

Rocq to Lean 4 Transition Guide

Generated for Pollux Project

December 31, 1979

Contents

1	Introduction	2
1.1	Philosophy Differences	3
2	Basic Syntax Translation	3
2.1	Definitions and Types	3
2.1.1	Simple Definitions	3
2.1.2	Let Bindings	3
2.1.3	Anonymous Functions	3
2.2	Inductive Types	4
2.2.1	Simple Inductive Types	4
2.2.2	Parameterized Inductive Types	4
2.2.3	Indexed Inductive Types (Dependent Types)	4
2.3	Records and Structures	5
2.4	Pattern Matching	5
2.5	Implicit Arguments	6
3	Type System Differences	6
3.1	Universe Hierarchy	6
3.2	Propositions vs Booleans	6
3.3	Equality	7
4	Theorems and Proofs	7
4.1	Stating Theorems	7
4.2	Proof Modes: Tactic vs Term	7
5	Tactic Translation Reference	8
5.1	Core Tactics	8
5.2	Induction Tactics	8
5.3	Rewriting	9
5.4	Working with Existentials	10
5.5	Automation	10
6	Structural Differences	11
6.1	Module System	11
6.2	Sections and Variables	12

6.3	Notations	13
6.4	Typeclasses	13
7	Proof Patterns for Verified Data Structures	14
7.1	Refinement Types / Subtypes	14
7.2	Decidable Propositions	14
7.3	Well-Founded Recursion	15
7.4	Mutual Recursion	15
7.5	Working with Binary Data (relevant to protobuf)	16
8	Standard Library Comparison	17
9	Project Setup	17
9.1	Creating a New Lean 4 Project	17
9.2	Project Structure	17
9.3	lakefile.lean (equivalent to _RocqProject + Makefile)	17
9.4	Building	18
10	Editor Setup	18
10.1	Emacs with lean4-mode	18
10.2	Key Bindings (lean4-mode)	18
11	Common Gotchas	18
11.1	1. Termination Checking	18
11.2	2. Definitional vs Propositional Equality	19
11.3	3. Coercions	19
11.4	4. Namespaces and Opening	19
11.5	5. Syntax Differences Summary	20
12	Resources	20
12.1	Official Documentation	20
12.2	For Coq/Rocq Users	20
12.3	Interactive Learning	20
12.4	Community	20

1 Introduction

This guide is designed for Rocq (Coq) users transitioning to Lean 4. It focuses on syntax mappings, tactic equivalents, and conceptual differences that will help you leverage your existing proof assistant experience.

1.1 Philosophy Differences

Aspect	Rocq	Lean 4
Foundation	CIC (Calculus of Inductive Constructions)	CIC variant with quotient types
Proof style	Tactic-focused	Term-mode + tactic hybrid
Extraction	Separate extraction mechanism	Native compilation to C
Universes	Set, Prop, Type	Prop, Type (unified)
Computation	Limited in proofs	Full computation everywhere

2 Basic Syntax Translation

2.1 Definitions and Types

2.1.1 Simple Definitions

```
(* Rocq *)
Definition x : nat := 5.
Definition f (n : nat) : nat := n + 1.

-- Lean 4
def x : Nat := 5
def f (n : Nat) : Nat := n + 1
```

2.1.2 Let Bindings

```
(* Rocq *)
Definition foo : nat :=
  let x := 5 in
  let y := x + 1 in
  x + y.

-- Lean 4
def foo : Nat :=
  let x := 5
  let y := x + 1
  x + y
```

2.1.3 Anonymous Functions

```
(* Rocq *)
Definition double := fun n => n * 2.
Definition add := fun x y => x + y.

-- Lean 4
def double := fun n => n * 2
def add := fun x y => x + y
-- Or using term-mode lambda
def add' : Nat → Nat → Nat := (· + ·)
```

2.2 Inductive Types

2.2.1 Simple Inductive Types

```
(* Rocq *)
Inductive day : Type :=
| monday
| tuesday
| wednesday
| thursday
| friday
| saturday
| sunday.

-- Lean 4
inductive Day : Type where
| monday
| tuesday
| wednesday
| thursday
| friday
| saturday
| sunday
```

2.2.2 Parameterized Inductive Types

```
(* Rocq *)
Inductive option (A : Type) : Type :=
| None : option A
| Some : A -> option A.

Inductive list (A : Type) : Type :=
| nil : list A
| cons : A -> list A -> list A.

-- Lean 4
inductive MyOption (a : Type) : Type where
| none : MyOption a
| some : a -> MyOption a

inductive MyList (a : Type) : Type where
| nil : MyList a
| cons : a -> MyList a -> MyList a
```

2.2.3 Indexed Inductive Types (Dependent Types)

```
(* Rocq *)
Inductive vec (A : Type) : nat -> Type :=
| vnil : vec A 0
| vcons : forall n, A -> vec A n -> vec A (S n).
```

```
-- Lean 4
inductive Vec (α : Type) : Nat → Type where
  | nil : Vec α 0
  | cons : α → Vec α n → Vec α (n + 1)
```

2.3 Records and Structures

```
(* Rocq *)
Record point := {
  x : nat;
  y : nat;
}.

Definition origin := { | x := 0; y := 0 | }.
Definition get_x (p : point) := p.(x).

-- Lean 4
structure Point where
  x : Nat
  y : Nat

def origin : Point := { x := 0, y := 0 }
-- Or using constructor syntax
def origin' : Point := ⟨0, 0⟩
def getX (p : Point) := p.x
```

2.4 Pattern Matching

```
(* Rocq *)
Definition is_zero (n : nat) : bool :=
  match n with
  | 0 => true
  | S _ => false
end.

Fixpoint length {A : Type} (l : list A) : nat :=
  match l with
  | nil => 0
  | cons _ t => S (length t)
end.

-- Lean 4
def isZero (n : Nat) : Bool :=
  match n with
  | 0 => true
  | _ + 1 => false

def length {α : Type} (l : List α) : Nat :=
  match l with
```

```
| [] => 0
| _ :: t => 1 + length t
```

2.5 Implicit Arguments

```
(* Rocq *)
Definition id {A : Type} (x : A) : A := x.
(* Explicit application *)
Check @id nat 5.
(* Implicit application *)
Check id 5.

-- Lean 4
def id {α : Type} (x : α) : α := x
-- Explicit application
#check @id Nat 5
-- Implicit application
#check id 5
-- Instance implicit (for typeclass resolution)
def show {α : Type} [ToString α] (x : α) : String := toString x
```

3 Type System Differences

3.1 Universe Hierarchy

```
(* Rocq: Three main sorts *)
Check nat.      (* nat : Set *)
Check Set.      (* Set : Type *)
Check Prop.     (* Prop : Type *)
Check Type.     (* Type : Type (universe polymorphism) *)

-- Lean 4: Two main sorts
#check Nat      -- Nat : Type
#check Type     -- Type : Type 1
#check Prop     -- Prop : Type
-- Prop is proof-irrelevant and impredicative
```

3.2 Propositions vs Booleans

```
(* Rocq: Prop is separate from bool *)
Definition is_even_prop (n : nat) : Prop := exists k, n = 2 * k.
Definition is_even_bool (n : nat) : bool := Nat.even n.

-- Lean 4: Prop and Bool are related via Decidable
def isEvenProp (n : Nat) : Prop := ∃ k, n = 2 * k
def isEvenBool (n : Nat) : Bool := n % 2 == 0

-- Decidable propositions can be used in if-then-else
def example1 (n : Nat) : Nat := if n < 5 then 0 else 1
```

3.3 Equality

```
(* Rocq *)
Check @eq.      (* eq : forall A : Type, A -> A -> Prop *)
Check @eq_refl. (* eq_refl : forall (A : Type) (x : A), x = x *)

-- Lean 4
#check @Eq      -- Eq :  $\alpha \rightarrow \alpha \rightarrow \text{Prop}$ 
#check @Eq.refl -- Eq.refl :  $\forall (a : \alpha), a = a$ 
-- Also heterogeneous equality
#check @HEq      -- HEq :  $\{\alpha : \text{Sort } u\} \rightarrow \alpha \rightarrow \{\beta : \text{Sort } u\} \rightarrow \beta \rightarrow \text{Prop}$ 
```

4 Theorems and Proofs

4.1 Stating Theorems

```
(* Rocq *)
Theorem plus_0_n : forall n : nat, 0 + n = n.
Proof. intros n. reflexivity. Qed.

Lemma plus_comm : forall n m : nat, n + m = m + n.
Proof. intros n m. induction n. ... Qed.

-- Lean 4
theorem plus_0_n :  $\forall n : \text{Nat}, 0 + n = n$  := by
  intro n
  rfl

lemma plus_comm :  $\forall n m : \text{Nat}, n + m = m + n$  := by
  intro n m
  induction n with
  | zero => simp
  | succ n ih => simp [Nat.succ_add, ih]

-- Or in term mode
theorem plus_0_n' (n : Nat) :  $0 + n = n$  := rfl
```

4.2 Proof Modes: Tactic vs Term

```
-- Lean 4: Term-mode proof
theorem and_comm_term {p q : Prop} (h : p  $\wedge$  q) : q  $\wedge$  p :=
  <h.2, h.1>

-- Lean 4: Tactic-mode proof
theorem and_comm_tactic {p q : Prop} (h : p  $\wedge$  q) : q  $\wedge$  p := by
  constructor
  · exact h.2
  · exact h.1
```

```
-- Lean 4: Mixed mode
theorem and_comm_mixed {p q : Prop} (h : p ∧ q) : q ∧ p := by
  exact ⟨h.2, h.1⟩
```

5 Tactic Translation Reference

5.1 Core Tactics

Rocq	Lean 4	Notes
intros x y z	intro x y z	Singular in Lean
reflexivity	rfl	Term or tactic
assumption	assumption	Same
exact H	exact h	Same
apply H	apply h	Same
rewrite H	rw [h]	Uses list syntax
rewrite ← H	rw [← h]	Backwards rewrite
simpl	simp	Much more powerful in Lean
unfold f	unfold f	Or simp only [f]
destruct x	cases x	Pattern match on hypothesis
induction x	induction x	With structured with syntax
split	constructor	Or And.intro/⟨_, _⟩
left/right	left/right	Same for disjunctions
exists t	use t	Or ⟨t, _⟩
trivial	trivial	Same
auto	simp_all	Or aesop
omega	omega	Linear arithmetic
lia	omega	Same solver
ring	ring	Requires import Mathlib.Tactic.Ring
field	field_simp	Requires Mathlib
discriminate	contradiction	Or simp often handles this
injection H	injection h	Or pattern match
inversion H	cases h	With auto-generated equations
assert (H : P)	have h : P : _ =	Or have h : P : by _ =
pose proof H as X	have x : h =	Introduce new hypothesis
specialize H x	have h : h x =	Or specialize h x
generalize x	generalize x	Similar
clear H	clear h	Same
rename H into H'	rename_i h'	Slightly different syntax
exfalso	exfalso	Same
subst x	subst x	Same
symmetry	symm	Same
transitivity y	trans y	Or calc blocks
congruence	congr	Similar
f_equal	congr_arg	As term or tactic

5.2 Induction Tactics

(* Rocq *)


```

Theorem plus_n_0 : forall n, n + 0 = n.
Proof.
  induction n as [| n' IHn'].
  - reflexivity.
  - simpl. rewrite IHn'. reflexivity.
Qed.

-- Lean 4
theorem plus_n_0 : ∀ n, n + 0 = n := by
  intro n
  induction n with
  | zero => rfl
  | succ n' ih => simp [Nat.succ_add, ih]

-- Alternative with induction'
theorem plus_n_0' : ∀ n, n + 0 = n := by
  intro n
  induction n
  case zero => rfl
  case succ n' ih => simp [Nat.succ_add, ih]

```

5.3 Rewriting

```

(* Rocq *)
Theorem example : forall a b c, a = b -> b = c -> a = c.
Proof.
  intros a b c Hab Hbc.
  rewrite Hab.
  rewrite Hbc.
  reflexivity.
Qed.

(* Multiple rewrites *)
Theorem example2 : forall a b, a = b -> a + a = b + b.
Proof.
  intros a b H.
  rewrite H.
  reflexivity.
Qed.

-- Lean 4
theorem example1 : ∀ a b c, a = b → b = c → a = c := by
  intro a b c hab hbc
  rw [hab, hbc]

-- Or using calc blocks (very powerful in Lean)
theorem example1' (a b c : α) (hab : a = b) (hbc : b = c) : a = c :=
  calc a = b := hab
    _ = c := hbc

```

```

theorem example2 :  $\forall a b, a = b \rightarrow a + a = b + b$  := by
  intro a b h
  rw [h]

-- simp can often handle rewrites automatically
theorem example2' :  $\forall a b, a = b \rightarrow a + a = b + b$  := by
  intro a b h
  simp [h]

```

5.4 Working with Existentials

```

(* Rocq *)
Theorem exists_example : exists n : nat, n = 0.
Proof.
  exists 0. reflexivity.
Qed.

Theorem exists_elim : (exists n, n = 0) -> True.
Proof.
  intros [n Hn]. trivial.
Qed.

-- Lean 4
theorem exists_example :  $\exists n : \text{Nat}, n = 0$  := by
  use 0

-- Term mode
theorem exists_example' :  $\exists n : \text{Nat}, n = 0$  := ⟨0, rfl⟩

theorem exists_elim : ( $\exists n, n = 0$ )  $\rightarrow$  True := by
  intro ⟨n, hn⟩
  trivial

-- Or using obtain
theorem exists_elim' : ( $\exists n, n = 0$ )  $\rightarrow$  True := by
  intro h
  obtain ⟨n, hn⟩ := h
  trivial

```

5.5 Automation

```

(* Rocq *)
Theorem auto_example : forall P Q : Prop, P -> (P -> Q) -> Q.
Proof. auto. Qed.

(* With hints *)
Theorem auto_with_hints : forall n, n + 0 = n.

```

```

Proof. intro. auto with arith. Qed.

-- Lean 4: simp is the workhorse
theorem simp_example :  $\forall P Q : \text{Prop}, P \rightarrow (P \rightarrow Q) \rightarrow Q :=$  by
  intro P Q hp hpq
  exact hpq hp

-- simp with lemmas
theorem simp_with_lemmas (n : Nat) :  $n + 0 = n :=$  by simp

-- aesop: general-purpose automation (like auto but better)
theorem aesop_example :  $\forall P Q : \text{Prop}, P \rightarrow (P \rightarrow Q) \rightarrow Q :=$  by aesop

-- decide: for decidable propositions
theorem decide_example :  $2 + 2 = 4 :=$  by decide

-- native_decide: uses kernel computation
theorem native_example :  $2 + 2 = 4 :=$  by native_decide

```

6 Structural Differences

6.1 Module System

```

(* Rocq: Module and Module Type *)
Module Type STACK.
  Parameter t : Type -> Type.
  Parameter empty : forall A, t A.
  Parameter push : forall A, A -> t A -> t A.
  Parameter pop : forall A, t A -> option (A * t A).
End STACK.

Module ListStack : STACK.
  Definition t (A : Type) := list A.
  Definition empty {A} : t A := nil.
  Definition push {A} (x : A) (s : t A) : t A := cons x s.
  Definition pop {A} (s : t A) : option (A * t A) :=
    match s with
    | nil => None
    | cons x xs => Some (x, xs)
    end.
End ListStack.

-- Lean 4: Structures/Classes and Namespaces
class Stack (t : Type → Type) where
  empty : t a
  push : a → t a → t a
  pop : t a → Option (a × t a)

```

```

instance : Stack List where
  empty := []
  push x s := x :: s
  pop
    | [] => none
    | x :: xs => some (x, xs)

-- Namespaces for organization
namespace MyModule
  def helper : Nat := 42
end MyModule

#check MyModule.helper

```

6.2 Sections and Variables

```

(* Rocq *)
Section WithA.
  Variable A : Type.
  Variable default : A.

  Definition get_or_default (o : option A) : A :=
    match o with
    | Some x => x
    | None => default
    end.
End WithA.

Check get_or_default.
(* get_or_default : forall A : Type, A -> option A -> A *)

-- Lean 4: variable command
variable (α : Type) (default : α)

def getOrElseDefault (o : Option α) : α :=
  match o with
  | some x => x
  | none => default

#check @getOrElseDefault
-- getOrElseDefault : (α : Type) → α → Option α → α

-- Or use sections
section WithA
  variable (α : Type) (default : α)

  def getOrElseDefault' (o : Option α) : α :=
    match o with

```

```

      | some x => x
      | none => default
end WithA

```

6.3 Notations

```

(* Rocq *)
Notation "x :: l" := (cons x l) (at level 60, right associativity).
Notation "[ ]" := nil.
Notation "[ x ; y ; .. ; z ]" := (cons x (cons y .. (cons z nil) ..)).

Infix "++" := app (at level 60, right associativity).

-- Lean 4: Multiple notation mechanisms
-- Infix notation
infixl:65 " ++ " => List.append

-- Prefix/Postfix
prefix:100 "!" => not
postfix:max "?" => Option

-- Bracket notation using macros
syntax "[" term,* "]" : term
macro_rules
  | `([$xs,*]) => `(List.cons $xs* List.nil)

-- More complex notations use macro system
notation "⟦" x "⟧" => singleton x

-- Scoped notations
scoped notation "∅" => EmptyCollection.emptyCollection

```

6.4 Typeclasses

```

(* Rocq *)
Class Eq (A : Type) := {
  eq : A -> A -> bool;
  eq_refl : forall x, eq x x = true
}.

Instance natEq : Eq nat := {
  eq := Nat.eqb;
  eq_refl := Nat.eqb_refl
}.

Definition member {A} {Eq A} (x : A) (l : list A) : bool :=
  existsb (eq x) l.

-- Lean 4: Very similar but cleaner syntax

```

```

class MyEq (α : Type) where
  eq : α → α → Bool
  eq_refl : ∀ x, eq x x = true

instance : MyEq Nat where
  eq := (· == ·)
  eq_refl := by intro x; simp [BEq.beq]

def member {α : Type} [MyEq α] (x : α) (l : List α) : Bool :=
  l.any (MyEq.eq x)

-- Lean has many built-in classes
#check (inferInstance : BEq Nat)
#check (inferInstance : Ord Nat)
#check (inferInstance : ToString Nat)

```

7 Proof Patterns for Verified Data Structures

Since you're working on data descriptor verification, here are some relevant patterns:

7.1 Refinement Types / Subtypes

```

(* Rocq: Using sig types *)
Definition positive := { n : nat | n > 0 }.

Program Definition safe_pred (p : positive) : nat :=
  match p with
  | exist _ n _ => n - 1
  end.

-- Lean 4: Subtype notation
def Positive := { n : Nat // n > 0 }

def safePred (p : Positive) : Nat := p.val - 1

-- Or using abbrev for transparency
abbrev Positive' := Subtype (· > 0)

-- Creating subtypes
def two : Positive := ⟨2, by decide⟩

```

7.2 Decidable Propositions

```

(* Rocq *)
Definition nat_eq_dec : forall n m : nat, {n = m} + {n <> m}.
Proof. decide equality. Defined.

Definition list_eq_dec {A} (eq_dec : forall x y : A, {x = y} + {x <> y})

```

```

    : forall l1 l2 : list A, {l1 = l2} + {l1 <> l2}.
Proof. decide equality. Defined.

-- Lean 4: Decidable typeclass
#check (inferInstance : DecidableEq Nat)
#check (inferInstance : DecidableEq (List Nat))

-- Custom decidable instance
instance : Decidable (n < m) := Nat.decLt n m

-- Using decidability in proofs
theorem example_dec (n m : Nat) : n = m ∨ n ≠ m := by
  by_cases h : n = m
  · left; exact h
  · right; exact h

```

7.3 Well-Founded Recursion

```

(* Rocq: Using Program or Equations *)
From Equations Require Import Equations.

Equations div (n m : nat) (H : m > 0) : nat by wf n lt :=
  div n m H with Nat.ltb n m := {
    | true => 0
    | false => S (div (n - m) m H)
  }.

-- Lean 4: termination_by clause
def div (n m : Nat) (h : m > 0) : Nat :=
  if n < m then 0
  else div (n - m) m h + 1
termination_by n
decreasing_by
  simp_wf
  omega

-- Using well-founded recursion explicitly
def ackermann : Nat → Nat → Nat
  | 0, n => n + 1
  | m + 1, 0 => ackermann m 1
  | m + 1, n + 1 => ackermann m (ackermann (m + 1) n)
termination_by m n => (m, n)

```

7.4 Mutual Recursion

```

(* Rocq *)
Fixpoint even (n : nat) : bool :=
  match n with
  | 0 => true

```

```

    | S n' => odd n'
  end
with odd (n : nat) : bool :=
  match n with
  | 0 => false
  | S n' => even n'
  end.
-- Lean 4
mutual
  def even : Nat → Bool
    | 0 => true
    | n + 1 => odd n

  def odd : Nat → Bool
    | 0 => false
    | n + 1 => even n
end

```

7.5 Working with Binary Data (relevant to protobuf)

```

-- Lean 4: ByteArray for binary data
def encodeVarint (n : Nat) : ByteArray := Id.run do
  let mut result := ByteArray.empty
  let mut value := n
  while value ≥ 128 do
    result := result.push ((value % 128 + 128).toUInt8)
    value := value / 128
  result := result.push value.toUInt8
  return result

-- Bitvectors (fixed-width)
#check (5 : BitVec 32) -- 32-bit bitvector

-- Proving properties about byte operations
theorem encodeVarint_nonempty (n : Nat) : (encodeVarint n).size > 0 := by
  simp [encodeVarint]
  sorry -- actual proof would depend on implementation

```


8 Standard Library Comparison

Rocq (stdpp/stdlib)	Lean 4 (Std/Mathlib)
list	List
option	Option
nat	Nat
Z	Int
bool	Bool
string	String
gmap (stdpp)	Std.HashMap
gset (stdpp)	Std.HashSet
fin n	Fin n
vec A n	Vector α n (Std)
byte	UInt8
word.word64 (Perennial)	UInt64 or BitVec 64

9 Project Setup

9.1 Creating a New Lean 4 Project

```
# Install elan (Lean version manager)
curl https://raw.githubusercontent.com/leanprover/elan/master/elan-init.sh -
sSf | sh
```

```
# Create new project
lake new myproject
```

```
# Or for a math project with Mathlib
lake new myproject math
```

9.2 Project Structure

```
myproject/
├─ lakefile.lean      # Build configuration (like _RocqProject + Makefile)
├─ lean-toolchain     # Lean version
├─ MyProject.lean     # Root file
└─ MyProject/
    ├─ Basic.lean
    └─ Advanced.lean
```

9.3 lakefile.lean (equivalent to _RocqProject + Makefile)

```
import Lake
open Lake DSL
```

```
package myproject where
```

```

-- Package configuration

@[default_target]
lean_lib MyProject where
  -- Library configuration

-- Add dependencies
require mathlib from git
  "https://github.com/leanprover-community/mathlib4"

```

9.4 Building

```

# Build project
lake build

# Update dependencies
lake update

# Clean
lake clean

```

10 Editor Setup

10.1 Emacs with lean4-mode

```

;; Install lean4-mode from MELPA
(use-package lean4-mode
  :straight (lean4-mode
    :type git
    :host github
    :repo "leanprover/lean4-mode"
    :files ("*.el" "data"))
  :commands lean4-mode)

```

10.2 Key Bindings (lean4-mode)

Key	Action	Rocq Equivalent
C-c C-k	Check file	C-c C-n (step)
C-c C-x	Execute command	C-c C-RET
C-c C-d	Show documentation	C-c C-a C-d
C-c C-g	Go to definition	M-.
C-c C-p	Show goal	C-c C-a C-s

11 Common Gotchas

11.1 1. Termination Checking

Rocq's `Fixpoint` is more lenient; Lean requires explicit termination proofs:

```
-- This might not pass termination checker without help
def f (n : Nat) : Nat :=
  if n = 0 then 0
  else f (n - 1) + 1
termination_by n -- explicit termination measure
```

11.2 2. Definitional vs Propositional Equality

Lean computes more aggressively; some Rocq `simpl` steps happen automatically:

```
-- This is definitionally true in Lean
example : 2 + 2 = 4 := rfl

-- But some reductions still need tactics
example (n : Nat) : n + 0 = n := by simp -- not rfl!
```

11.3 3. Coercions

```
(* Rocq: Explicit coercions *)
Coercion nat_to_Z : nat >-> Z.

-- Lean 4: Instance-based coercions
instance : Coe Nat Int where
  coe := Int.ofNat

-- Or using ↑ notation explicitly
#check (↑5 : Int) -- 5 coerced from Nat to Int
```

11.4 4. Namespaces and Opening

```
-- Lean uses dot notation extensively
#check List.map -- full name
#check [1,2,3].map -- method syntax

-- Opening namespaces
open List in
#check map -- now just `map` works

-- Scoped opening
open List
#check map
```

11.5 5. Syntax Differences Summary

Concept	Rocq	Lean 4
Function arrow	$\rightarrow$ or $\rightarrow$	$\rightarrow$
Product type	$*$	$\times$
Lambda	$\text{fun } x \Rightarrow$	$\text{fun } x \Rightarrow$
Forall	forall	$\forall$
Exists	exists	$\exists$
Not equal	$\neq$	$\neq$
And (Prop)	$\wedge$	$\wedge$
Or (Prop)	$\vee$	$\vee$
Iff	$\leftrightarrow$	$\leftrightarrow$
Comment	$(* \dots *)$	$-- \dots / - \dots - /$
String	$" \dots "$	$" \dots "$

12 Resources

12.1 Official Documentation

- Lean 4 Documentation
- Theorem Proving in Lean 4 (the main tutorial)
- Functional Programming in Lean

12.2 For Coq/Rocq Users

- Mathlib4 - The main mathematics library
- Mathlib4 Documentation

12.3 Interactive Learning

- Natural Number Game - Interactive tutorial

12.4 Community

- Lean Zulip Chat - Very active and helpful
- Mathlib GitHub